You convert these examples into a slightly more structured format ❸, known as scenarios, so that you can automate them in the form of automated acceptance tests. Naturally, this test should fail at first ❹, because you haven't implemented anything yet. The developers now proceed to write the code required to make this acceptance criterion pass ❺. The developer uses TDD, or more precisely, BDD at a unit-testing level, to implement the various layers needed to deliver the requested functionality. We'll discuss how to write and automate these scenarios in the next chapter.

If the developer has worked well, the acceptance criterion should now pass ❻, which indicates that you're done with this scenario and can move on to the next one ❼. When all of the scenarios pass, the developers are done with this story ❽. QA can now use the passing automated acceptance tests as a basis for their exploratory testing, and stakeholders can review the new feature to see how well it matches their expectations.

4.6 Summary

In this chapter, you learned how to describe and organize features and how to illustrate them with examples. Along the way, you learned about the following:

- A *capability* enables some business goal, regardless of implementation.
- A *feature* is a piece of deliverable software functionality that provides users with a capability.
- Large features can be broken down into smaller features to make them easier to organize and deliver.
- Agile projects use *user stories* to plan and deliver features.
- Concrete examples help you build up a shared understanding about a feature.
- The principle of *Real Options* recommends that you shouldn't commit to a particular solution until you have enough information to be confident that it's the most appropriate one.
- *Deliberate Discovery* points out that one of the biggest risks in any software project is your own ignorance, and that you should actively aim to identify and reduce uncertainty wherever you can.

A surprising number of the benefits of BDD come from simply having a conversation with the business, using examples to challenge assumptions and build a common understanding of the problem space. One of the principle benefits of BDD is to encourage and structure this kind of conversation. But there's also a great deal to gain by automating these examples, in the form of automated acceptance criteria. In the next chapter, you'll learn how to express clear, precise examples in a structured format, and how to turn these examples into executable specifications that can be read by tools like JBehave, Cucumber, and SpecFlow.